

Layer 1 — Runtime substrate

Product-owner version

Product goal: create a reliable machine and remote-model foundation before any research, publishing, Agentfield, or Paperclip automation is built.

Layer 1 answers:

Can I start from a clean local/WSL/Runpod runtime and know where everything lives?

Can I verify workspace, GPU, CUDA, Docker, and remote model readiness without accidentally launching exp

Can local code call a remote model through one stable contract?

Product boundary

Layer 1 prepares the **runtime substrate** only.

It should do:

- prepare workspace roots
- prepare persistent volume layout
- check GPU/CUDA/PyTorch readiness
- check Docker/remote-compute readiness
- prepare a thin remote model client contract

It should not do:

- not create the NCA-ART-GRN research repo
- not run simulations
- not train models
- not build Agentfield

```
not build Paperclip adapter
not index PKM
not build papers
not launch Runpod jobs by default
```

Bundles inside Layer 1

Bundle 1 – Runpod portable runtime base
Bundle 7 – Remote model brain endpoint

Bundle 1 — Runpod portable runtime base

Product outcome: a machine or pod has predictable paths, runtime checks, and storage roots.

Concretizations:

```
prepare_runpod_workspace
check_runpod_workspace
check_gpu_runtime
check_cuda_torch_runtime
prepare_runpod_volume_layout
prepare_docker_runtime_policy
check_docker_gpu_access
prepare_remote_compute_profile
```

Generic workspace roots:

```
/workspace/repos
/workspace/data
/workspace/runs
/workspace/artifacts
```

```
/workspace/models
```

Project-specific folders come later, for example:

```
/workspace/repos/nca-art-grn  
/workspace/data/nca-art-grn  
/workspace/runs/nca-art-grn  
/workspace/artifacts/nca-art-grn
```

Product value: prevents manual Runpod drift. Docker gives the stable base; config prepares users, roles, environments, paths, and policy after pod start.

Bundle 7 — Remote model brain endpoint

Product outcome: one minimal, stable loop:

```
local code -> remote model -> response
```

Concretizations:

```
prepare_remote_model_client  
check_runpod_brain_endpoint  
prepare_brain_router_project  
check_opencode_remote_model_config
```

This creates or verifies:

```
runpod_brain_client.py  
brain_router.py  
prompts.py  
smoke_test.py
```

Product value: OpenCode, OpenClaw, Agentfield reasoners, and later agents can all reuse the same remote-model contract instead of each tool calling Runpod differently.

Layer 1 success condition

After Layer 1, you should be able to say:

The runtime has stable storage roots.

GPU/CUDA can be checked safely.

Docker/remote-compute readiness can be inspected.

A remote model endpoint can be called through one thin client.

No science or platform orchestration has been hidden inside setup.